Karpatkey Tokenized Fund

Côme du Crest

2024-10-25

Table of contents

- Table of contents
- Karpatkey Tokenized Fund Audit
  - Scope
  - Context
  - Status
  - Legal Information And Disclaimer
- Issues
  - [Med] No guarantee on ordering of id in EnumerableMaps
  - [Low] CompoundV3Comet fails when reward token is also base token
  - Optimisations and miscellaneous

Karpatkey Tokenized Fund Audit

This document presents the finding of a smart contract audit conducted by Côme du Crest for Gnosis.

Scope

The scope includes all contracts within karpatkey-tokenized-fund as of commit 6ae63d3. I also quickly browsed the backend code to see how contracts were used.

Context

The goal of the project is to hold funds on multiple chain invested in various protocols. A single `Shares` contract deployed on mainnet is responsible for users joining and exiting the protocol. Users join using USDC to receive shares tokens and exit their shares tokens back into a hopefully higher amount of USDC. In the meantime, USDC is re-invested by the operator for a profit. The join and exit rates are computed off-chain and validated by the operator at the time of join and exit.

To help compute the join and exit rates, `NAVCalculator` contracts are deployed on each chain that aggregate the handled tokens balances (e.g. Aave tokens, Compound tokens, etc ...) and query oracles for the price of the tokens to compute the total value held on that chain. The total value held by the protocol is the sum of the NAV values computed on each chain.

To prevent double counting of assets during bridging, the operator will signal when a bridge transfer is taking place and when it ends. The `NAVCalculator` will return a non-zero `issues` value when queried if a bridge operation is pending, signaling that the calculated NAV is incorrect.

The protocol relies on a complete trust of the admin that can upgrade the `Shares` contract and the operator that can charge any amount of fees they like.

Status

The report has been sent to the core developer.

Fixes were implemented in commit faa34bd.

Legal Information And Disclaimer

1. This report is based solely on the information provided by Karpatkey (the “Company”), with the assumption that the information provided to Gnosis is authentic, accurate, complete, and not misleading as of the date of this report. Gnosis has not conducted any independent enquiries, investigations or due diligence in respect of the Company, its business or its operations.
2. Changes to the information contained in the documents, repositories and any other materials referenced in this report might affect or change the analysis and conclusions presented. Gnosis is not responsible for monitoring, nor will we be aware of, any future additions, modifications, or deletions to the audited code. As such, Gnosis does not assume any responsibility to update any information, content or data contained in this report following the date of its publication.
3. This report does not address, nor should it be interpreted as addressing, any regulatory, tax or legal matters, including but not limited to: tax treatment, tax consequences, levies, duties, data privacy, data protection laws, issues relating to the licensing of information technology, intellectual property, money laundering and countering the financing of terrorism, or any other legal restrictions or prohibitions. Gnosis disclaims any liability for omissions or errors in the findings or conclusions presented in this report.
4. The views expressed in this report are solely our views regarding the specific issues discussed within this report. This report is not intended to be exhaustive, nor should it be construed as an assurance, guarantee or warranty that the code is free from bugs, vulnerabilities, defects or deficiencies. Different use cases may carry different risks, and integration with third-party applications may introduce additional risks.
5. This report is provided for informational purposes only and should not be used as the basis for making investment or financial decisions. This report does not constitute investment research and should not be viewed as an invitation, recommendation, solicitation or offer to subscribe for or purchase any securities, investments, products or services. Gnosis is not a financial advisor, and this report does not constitute financial or investment advice.
6. The statements in this report should be considered as a whole, and no individual statement should be extracted or referenced independently.
7. This report is addressed exclusively to the Company, and except with prior written consent from Gnosis, it may not be shared, disclosed, transmitted, or relied upon by any other person or entity for any purpose.
8. To the fullest extent permitted by applicable laws, Gnosis disclaims any and all other liability, whether in contract, tort, or otherwise, that may arise from this report or the use thereof.

Issues

[Med] No guarantee on ordering of id in EnumerableMaps

Summary

The `Balances` and `NAVCalculator` contracts use OpenZeppelin's `EnumerableMap` library to store the balance oracles and asset decimals. The library does not guarantee that ordering of elements remains the same after adding or deleting elements. However, both contracts expose function that takes as an input a position of an element in the map. If the user is not cautious they could expect to refer to a certain element at a position while that element changed position.

Vulnerability Detail

`EnumerableMap` does not guarantee the ordering of its elements:

```
    /**
     * @dev Returns the element stored at position `index` in the map. O(1).
     * Note that there are no guarantees on the ordering of values inside the
     * array, and it may change when more values are added or removed.
     *
     * Requirements:
     *
     * - `index` must be strictly less than {length}.
     */
    function at(AddressToUintMap storage map, uint256 index) internal view
        returns (address, uint256) {
        ...
    }
```

`Balances.balanceOf()` uses `id` as an input variable to get the balance of a token using oracle at position `id`:

```
    function balanceOf(address token, uint256 idx, address safe) public view
        returns (int256) {
        if (idx >= _balanceOracles[token].length()) {
            revert BalanceOracleNotSet();
        }

        (address oracleAddress, uint256 balanceTypeUInt) = _balanceOracles[
            token].at(idx);  // @audit there is no guarantee on ordering of idx
        ...
    }
```

`NAVCalculator.readPositionsRange()` reads a subset of assets by iterating over a limited number of asset decimals:

```
    function readPositionsRange(address safe, uint256 from, uint256 to)
        public
        view
        returns (int256 usd, uint16 issues)
    {
        if (from > to || to >= _assetDecimals.length()) {
            revert InvalidArguments();
        }

        if (checkUptime()) {
            issues++;
        }

        // iterate over every known asset
        for (uint256 i = from; i <= to; i++) {
            (address assetAddr,) = _assetDecimals.at(i);  // @audit unsafe when
                 reading assets

            ...
        }
    }
```

`NavCalculator.readPosition()` similarly exposes a function to read a position of an asset using the id of a balance oracle:

```
    function readPosition(address safe, address assetAddr, uint256
        idxBalanceOracle)
        external
        view
        returns (INAVCalculator.Position memory p)
    {
        if (!_assetDecimals.contains(assetAddr)) {
            revert AssetNotFound();
        }

        if (priceOracles[assetAddr].oracle == address(0)) {
            revert PriceOracleNotSet();
        }

        bool bridging = isAssetBridging(assetAddr);

        (uint256 usdRate, uint256 usdRateDecimals, bool stale) = _latestPrice(
            assetAddr);

        if (checkUptime()) {
            stale = true;
        }

        return _readPositionWithPrice(safe, assetAddr, idxBalanceOracle,
            usdRate, usdRateDecimals, bridging, stale);  // @audit unsafe use of
             id (unused in backend)
    }
```

Impact

The id of elements of an enumerable map should only be used to enumerate (iterate) over all the elements of a map in a single transaction or extra care should be taken to make sure the correct element is referenced.

In the case of `Balances.balanceOf()` if a mapping `id => oracle` is stored off-chain to know the `id` to use to query for a balance, that mapping will eventually miss-algin with the on-chain mapping after adding and removing elements in the map. The caller of `balanceOf()` will expect the balance of an asset using a certain oracle and receive the balance using a different oracle. The problem is similar for `NavCalculator.readPosition()`.

In the case of `NAVCalculator.readPositionsRange()` if a user attempts to read positions of assets from `0` to `20` and decompose it in assets from `0` to `10` and `11` to `20` they may read the same asset twice as it can change position.

I believe these functions are currently unused in the backend but they could bring about a subtle bug that would lead to incorrect calculation of balances or NAV in the future.

Code Snippets

https://github.com/OpenZeppelin/openzeppelin-contracts/blob/28aed34dc5e025e61ea0390c18cac875bfde1a78/contracts/utils/structs/EnumerableMap.sol#L481-L493

https://github.com/karpatkey/karpatkey-tokenized-fund/blob/6ae63d373fc562c36916c43e3cfe032bcfd1950d/contracts/src/balances/Balances.sol#L59-L82

https://github.com/karpatkey/karpatkey-tokenized-fund/blob/6ae63d373fc562c36916c43e3cfe032bcfd1950d/contracts/src/nav/NAVCalculator.sol#L135-L199

Recommendation

Do not expose functions that use the underlying `id` for enumerable maps without ensuring the correct element is at the expected position. Otherwise (it may be necessary to expose `readPositionsRange`) write an explicit warning in the function documentation.

Response

Natspec comments have been added to the concerned functions in commit faa34bd.

[Low] CompoundV3Comet fails when reward token is also base token

Summary

The function `CompoundV3Comet.balanceOf(token, safe)` aggregates results from different function calls depending on whether the queried token is a reward, base, or collateral token. If the queried token is used both as reward and base token it will be handled only as a reward token and return an incorrect result.

Vulnerability Detail

`balanceOf()` dissociates the case where token is a reward token to the case where it is a base token:

```
1     function balanceOf(address token, address safe) external view returns (
          int256 balance) {
2         if (token != address(rewardToken) && token != baseToken && !
              isCollateral[token]) {
3             revert InvalidArguments();
4         }
5
6         if (token == address(rewardToken)) {
7             balance += _getRewardOwed(safe).toInt256();
8         } else if (token == baseToken) {
9             balance += getUnderlying(comet, safe);  // @audit a baseToken can
                  be a rewardToken
10        }
11
12        if (isCollateral[token]) {
13            balance += CometExtInterface(comet).collateralBalanceOf(safe, token
                  ).toInt256();
14        }
15
16        return balance;
17    }
```

Impact

To my understanding from reading the code of CompoundV3 and their documentation, I do not see a reason why a base token would not also be a reward token. If that ever becomes the case, the balance returned by `CompoundV3Comet` will be inaccurate.

This also affects `CompoundV3CometWithMultiplier` in the exact same way.

Code Snippets

https://etherscan.io/address/0x1B0e765F6224C21223AeA2af16c1C46E38885a40#code

https://github.com/karpatkey/karpatkey-tokenized-fund/blob/6ae63d373fc562c36916c43e3cfe032bcfd1950d/contracts/src/balances/protocols/CompoundV3Comet.sol#L111-L127

https://github.com/karpatkey/karpatkey-tokenized-fund/blob/6ae63d373fc562c36916c43e3cfe032bcfd1950d/contracts/src/balances/protocols/CompoundV3CometWithMultiplier.sol#L111-L127

Recommendation

If the developer believes that the base token cannot be a reward token, this can be left as is. Otherwise, replace the `else` block with a simple `if`.

Response

The `else if` clause has been replaced by a simple `if` in commit faa34bd.

Optimisations and miscellaneous

This part lists minor gas/code optimizations that shouldn't make the code less readable or improve overall readability. It also lists questions about unclear code segments or comments that do not warrant an issue.

No protection on changes of TTL for user of Shares

If the redeem or deposit request TTL is changed in between a user submitting a redeem or deposit request transaction and the transaction being applied, users could sign up for a different TTL than anticipated. This means the user will not be able to cancel their request before a potentially longer time than expected. I do not believe this is a worrying issue as I do not think this value will be changed often and trusted operator can cancel a redeem or deposit request during that time anyway.

Non-standard gap size

The different upgradeable contracts use a `uint256[50] private __gap` value to provision for the possible added storage variables in future versions. It is a good idea, however it is more common and expected to provision for `50` total storage slot per contract and thus set the size of the gap to `50 - already_allocated_storage_slots` so that each contract takes exactly `50` slots. This make it easier to analyze the storage layout of the contracts once deployed.

https://github.com/karpatkey/karpatkey-tokenized-fund/blob/6ae63d373fc562c36916c43e3cfe032bcfd1950d/contracts/src/nav/BridgeAware.sol#L124

https://github.com/karpatkey/karpatkey-tokenized-fund/blob/6ae63d373fc562c36916c43e3cfe032bcfd1950d/contracts/src/balances/Balances.sol#L154

NAVCalculator._toUsd() will not work when decimals + usdRateDecimals < NAV_DECIMALS

The function `NAVCalculator._toUsd()` takes into account decimal by dividing the result by `10 ** (decimals + usdRateDecimals - NAV_DECIMALS)` which will underflow when decimals + usdRateDecimals < NAV_DECIMALS. This is not expected to be a problem since `NAV_DECIMALS = 8`, `decimals` represent the decimals of the accounted asset, and `usdRateDecimals` is the decimals of price conversion usually involving an 8 decimals oracle.